

Bilan des travaux réalisés

- **Mise en route matérielle (Arduino Nano 33 IoT)** : Configuration de la puce radio NINA-B306 pour émettre un signal Bluetooth Low Energy (BLE) continu à 2,4 GHz de manière 100 % autonome dès la mise sous tension.
- **Acquisition haute fréquence (40 Hz)** : Création d'un tunnel de communication actif entre l'Arduino et Python sous Linux, permettant de capturer près de 600 échantillons RSSI en 15 secondes.
- **Traitement du signal (DSP)** : Implémentation d'une chaîne de filtrage numérique nettoyant le signal brut (filtre médian et lissage Savitzky-Golay) avant d'extraire l'onde respiratoire (filtre passe-bande de Butterworth entre 0,15 et 0,50 Hz) et la bande des micro-variations cardiaques (0,90 à 2,00 Hz).
- **Monitoring clinique en direct** : Développement d'une interface graphique temps réel à trois courbes glissantes (signal radio, respiration, bande cardiaque) avec enregistrement simultané des mesures dans un fichier CSV.
- **Validation expérimentale** : Détection nette des cycles d'inspiration et d'expiration (variations périodiques de 6 à 8 dB), et identification de la limite physique du capteur pour le rythme cardiaque à distance (la résolution matérielle de 1 dB du RSSI masque les micro-mouvements précordiaux inférieurs à 0,05 dB).

Problèmes rencontrés et solutions

- **Puce BLE muette et blocage au démarrage** : La bibliothèque standard ne trouvait pas le bus matériel et la ligne `while (!Serial);` bloquait la carte après un débranchement.
 - *Solution* : Inclusion forcée du bus `<SPI.h>`, utilisation de la source GitHub officielle dans PlatformIO et suppression de l'attente du port série.
- **Sous-échantillonnage critique sous Linux (1 à 3 points reçus en 15 s)** : En mode scan passif, le gestionnaire Bluetooth Linux (BlueZ) bloquait les paquets répétés via son filtre anti-spam, ne laissant qu'une seule ligne exploitable dans le fichier CSV initial.
 - *Solution* : Passage en mode connexion directe (`BleakClient`) avec création d'un service de streaming GATT (`BLENotify`). L'Arduino mesure lui-même son registre `BLE.rssi()` toutes les 25 ms et force l'envoi au PC.
- **Crashes liés aux mises à jour de la bibliothèque `bleak`** : Erreurs `AttributeError` sur `device.rssi` et suppression de la fonction `client.get_rssi()` dans les versions récentes de Python.
 - *Solution* : Décodage binaire direct (`struct.unpack`) des entiers 32-bits envoyés par l'Arduino via les notifications Bluetooth.
- **Pics d'erreurs radio (-105 dBm)** : Des collisions ponctuelles à 2,4 GHz faussaient le comportement des filtres de Butterworth.
 - *Solution* : Rejet par seuillage dès la réception et application d'un filtre médian avant le filtrage fréquentiel.
- **Lenteur d'affichage et courbes en « marches d'escalier »** : Le moteur `Matplotlib` bloquait la réception des données pendant près d'une seconde à chaque mise à jour des axes, regroupant les paquets au même horodatage et déformant les courbes en paliers rectangulaires

